

การทำเหมืองข้อมูล

รหัสวิชา 040513407-66

รองศาสตราจารย์ ดร.วิกานดา ผาพันธุ์

ภาควิชาสถิติประยุกต์ คณะวิทยาศาสตร์ประยุกต์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

พ.ศ. 2569

บทที่ 1 การเก็บรวบรวมข้อมูล (Data Collection)

วัตถุประสงค์การเรียนรู้

เมื่อศึกษาบทนี้แล้ว ผู้เรียนสามารถ

- อธิบายความสำคัญและบทบาทของการเก็บรวบรวมข้อมูลในกระบวนการทำเหมืองข้อมูล
- อ่านข้อมูลจากไฟล์รูปแบบต่างๆ เช่น CSV, Excel, JSON, XML และ HTML ด้วยไลบรารี Pandas ในโปรแกรมภาษา Python
- ดึงข้อมูลจากเว็บไซต์ที่มีโครงสร้างด้วย `pd.read_html()` และจากเว็บไซต์กึ่งโครงสร้างด้วยเทคนิค Web Scraping โดยใช้ BeautifulSoup
- เชื่อมต่อและสืบค้นข้อมูลจากฐานข้อมูล SQLite ด้วย Python
- ดึงข้อมูลทางการเงินจาก Yahoo Finance ผ่าน API โดยใช้ไลบรารี `yfinance`

1.1 บทนำ

การเก็บรวบรวมข้อมูล (Data Collection) เป็นจุดเริ่มต้นที่สำคัญที่สุดในกระบวนการทำเหมืองข้อมูล เพราะคุณภาพและความครบถ้วนของข้อมูลที่รวบรวมได้ย่อมส่งผลโดยตรงต่อความน่าเชื่อถือของผลการวิเคราะห์ในทุกขั้นตอนที่ตามมา ในโลกที่ข้อมูลถูกสร้างขึ้นอย่างต่อเนื่องจากแหล่งที่หลากหลาย ทั้งไฟล์ข้อมูล ฐานข้อมูล เว็บไซต์ และบริการ API จึงจำเป็นต้องมีเครื่องมือและเทคนิคที่เหมาะสมสำหรับแต่ละแหล่งข้อมูล

Python ได้รับความนิยมอย่างแพร่หลายในงานเหมืองข้อมูล ส่วนหนึ่งเป็นเพราะมีระบบนิเวศของไลบรารีที่ครอบคลุมทุกขั้นตอนการทำงาน ตั้งแต่การเข้าถึงข้อมูล การทำความสะอาด ไปจนถึงการวิเคราะห์และนำเสนอผล ไลบรารีหลักที่ใช้ในตอนนี้มีดังนี้

Pandas ไลบรารีสำหรับการจัดการและวิเคราะห์ข้อมูลที่มีโครงสร้างรองรับการนำเข้าข้อมูลจากไฟล์หลากหลายรูปแบบและจากฐานข้อมูล SQL

BeautifulSoup ไลบรารีสำหรับ Web Scraping ช่วยแยกวิเคราะห์เอกสาร HTML และ XML เพื่อดึงข้อมูลที่ต้องการออกมาได้อย่างมีประสิทธิภาพ

Requests ไลบรารีสำหรับส่ง HTTP Request ใช้โต้ตอบกับเว็บและ API โดยทำให้กระบวนการเชื่อมต่อเป็นเรื่องง่าย

mysql-connector-python, psycopg2, sqlite3 ตัวเชื่อมต่อฐานข้อมูลสำหรับ MySQL, PostgreSQL และ SQLite ตามลำดับ

yfinance ไลบรารีสำหรับเข้าถึงข้อมูลทางการเงินจาก Yahoo Finance รวมถึงราคาหุ้นย้อนหลังและข้อมูลบริษัท

1.2 ทฤษฎีพื้นฐานของเหมืองข้อมูล

การทำเหมืองข้อมูล (Data Mining) หมายถึงกระบวนการค้นพบรูปแบบที่มีประโยชน์และไม่เคยรู้จักมาก่อนจากชุดข้อมูลขนาดใหญ่ โดยอรรถาโนมัตติ (Tan et al., 2006) โดยเป็นส่วนหนึ่งของกระบวนการค้นพบความรู้ในฐานข้อมูล (Knowledge Discovery in Databases: KDD) ซึ่งครอบคลุมทุกขั้นตอนตั้งแต่การเตรียมและคัดเลือกข้อมูล การแปลงข้อมูล ไปจนถึงการตีความและประเมินผล

ในเชิงสังคมศาสตร์ที่ผ่านมากว่า ความก้าวหน้าของเทคโนโลยีทำให้องค์กรทั่วโลกสามารถสะสมข้อมูลได้ในปริมาณที่เพิ่มขึ้นอย่างรวดเร็ว ก้าวกระโดด

แต่เครื่องมือวิเคราะห์แบบดั้งเดิมกลับไม่สามารถรับมือกับข้อมูลที่มีขนาดใหญ่ มีหลายมิติ หรือ มี โครงสร้างที่ซับซ้อน ได้อย่างเพียงพอ ความต้องการดังกล่าวจึงเป็นแรงผลักดันสำคัญที่ทำให้สาขาวิชาการทำเหมืองข้อมูลเติบโตขึ้น

1.2.1 ต้นกำเนิดและสาขาวิชาที่เกี่ยวข้อง

การทำเหมืองข้อมูลเป็นสาขาวิชาที่เกิดจากการบูรณาการความรู้หลายแขนงเข้าด้วยกัน ได้แก่ สถิติ (Statistics) ปัญญาประดิษฐ์ (Artificial Intelligence) การเรียนรู้ของเครื่อง (Machine Learning) การจดจำรูปแบบ (Pattern Recognition) เทคโนโลยีฐานข้อมูล (Database Technology) และการประมวลผลแบบขนาน (Parallel/Distributed Computing)

Tan et al. (2006)

อธิบายว่าการทำเหมืองข้อมูลนำแนวคิดจากสถิติมาใช้ในการสุ่มตัวอย่าง การประมาณค่า และการทดสอบสมมติฐาน ขณะที่ยังดึงเอาอัลกอริทึมการค้นหา เทคนิคการรู้จำรูปแบบ และทฤษฎีการเรียนรู้จากสาขาปัญญาประดิษฐ์และการจดจำรูปแบบมาประยุกต์ใช้ควบคู่กัน

Hastie, Tibshirani และ Friedman (2009) มองจากมุมมองการเรียนรู้เชิงสถิติ (Statistical Learning) ว่ามีบทบาทสำคัญในวิทยาศาสตร์ การเงิน และอุตสาหกรรม โดยมีเป้าหมายร่วมกันคือการสร้างโมเดลที่สามารถทำนายผลลัพธ์สำหรับข้อมูลใหม่ได้อย่างแม่นยำ บนพื้นฐานของข้อมูลฝึก (Training Set) ที่รวบรวมไว้

1.2.2 ประเภทของการเรียนรู้ (Types of Learning)

Bishop (2006) แบ่งการเรียนรู้ในสาขาการจดจำรูปแบบและการเรียนรู้ของเครื่องออกเป็น 3 ประเภทหลัก ดังนี้

Supervised Learning (การเรียนรู้แบบมีผู้สอน) เป็นการเรียนรู้จากชุดข้อมูลฝึกที่มีทั้งข้อมูลนำเข้า (Input Vectors) และค่าเป้าหมาย (Target Values) กำกับไว้ครบถ้วน แบ่งย่อยออกเป็นสองกลุ่ม ได้แก่ Classification

สำหรับจำแนกข้อมูลออกเป็นกลุ่มที่กำหนด และ Regression สำหรับทำนายค่าต่อเนื่อง ตัวอย่างเช่น การทำนายราคาหุ้น การวินิจฉัยโรค และการกรองอีเมลขยะ

Unsupervised Learning (การเรียนรู้แบบไม่มีผู้สอน) เป็นการเรียนรู้จากข้อมูลที่ไม่มีความหมายกำกับ โดยมุ่งค้นหาโครงสร้างหรือรูปแบบแฝงในข้อมูล ตัวอย่างงานในกลุ่มนี้ได้แก่ การจัดกลุ่ม (Clustering) การประมาณการแจกแจงความน่าจะเป็น (Density Estimation) และการลดจำนวนมิติ (Dimensionality Reduction) เพื่อนำเสนอข้อมูลในรูปที่เข้าใจง่ายขึ้น

Reinforcement Learning (การเรียนรู้แบบเสริมแรง) เป็นการเรียนรู้ที่ระบบไม่ได้รับตัวอย่างคำตอบที่ถูกต้องโดยตรง แต่ต้องค้นพบแนวทางที่ดีที่สุดผ่านการลองผิดลองถูก (Trial and Error) โดยใช้รางวัล (Reward) เป็นสัญญาณป้อนกลับ เหมาะกับปัญหาที่ต้องการตัดสินใจต่อเนื่อง เช่น การเล่นเกมหรือการควบคุมระบบอัตโนมัติ

1.2.3 งานหลักของเหมืองข้อมูล (Core Data Mining Tasks)

Tan et al. (2006) จำแนกงานของการทำเหมืองข้อมูลออกเป็น 2 กลุ่มใหญ่ตามวัตถุประสงค์

Predictive Tasks (งานเชิงทำนาย) มีเป้าหมายเพื่อทำนายค่าของตัวแปรเป้าหมาย (Target Variable หรือ Dependent Variable) จากตัวแปรอื่นๆ ที่เรียกว่าตัวแปรอธิบาย (Explanatory Variables หรือ Independent Variables)

Descriptive Tasks (งานเชิงพรรณนา) มีเป้าหมายเพื่อค้นพบรูปแบบที่สะท้อนความสัมพันธ์พื้นฐานในข้อมูล เช่น ความสัมพันธ์ (Correlation) แนวโน้ม (Trends) กลุ่ม (Clusters) และความผิดปกติ (Anomalies) งานกลุ่มนี้มักมีลักษณะสำรวจข้อมูล (Exploratory) และต้องการการตีความผลลัพธ์เพิ่มเติมหลังการประมวลผล

ภายใต้สองกลุ่มนี้ มีงานหลัก 4 ประเภทที่พบบ่อยในภาคปฏิบัติ ได้แก่ (1) Predictive Modeling ซึ่งครอบคลุม Classification และ Regression (2) Association Analysis

สำหรับค้นหาความสัมพันธ์ระหว่างตัวแปร (3) Cluster Analysis สำหรับจัดกลุ่มข้อมูลที่มีลักษณะใกล้เคียงกัน และ (4) Anomaly Detection สำหรับระบุข้อมูลที่ผิดปกติออกจากรูปแบบทั่วไป

1.2.4 กระบวนการค้นพบความรู้ในฐานข้อมูล (KDD Process)

กระบวนการค้นพบความรู้ในฐานข้อมูล (Knowledge Discovery in Databases: KDD) คือกรอบการทำงานที่ครอบคลุมทุกขั้นตอนในการแปลงข้อมูลดิบให้กลายเป็นความรู้ที่นำไปใช้ได้จริง ประกอบด้วย 4 ขั้นตอนหลัก ดังนี้

(1) **Input Data** ข้อมูลดิบจากแหล่งต่างๆ เช่น ไฟล์ข้อความ สเปรดชีต หรือตารางในฐานข้อมูลสัมพันธ์ ซึ่งอาจอยู่ในรูปแบบรวมศูนย์หรือกระจายตามแหล่งต่างๆ

(2) **Data Preprocessing** การเตรียมข้อมูลให้พร้อมสำหรับการวิเคราะห์ ครอบคลุมการคัดเลือกคุณลักษณะ (Feature Selection) การลดจำนวนมิติ (Dimensionality Reduction) การปรับมาตรฐาน (Normalization) และ การกำหนดขอบเขตข้อมูล (Data Subsetting) ขั้นตอนนี้มักใช้เวลาามากที่สุดในกระบวนการ KDD ทั้งหมด

(3) **Data Mining** การนำอัลกอริทึมมาใช้ค้นหารูปแบบหรือความรู้จากข้อมูลที่เตรียมไว้แล้ว ถือเป็นหัวใจหลักของกระบวนการ KDD

(4) **Postprocessing** การกรองและประเมินรูปแบบที่ค้นพบ รวมถึงการแสดงผลด้วยภาพ (Visualization) และการตีความเพื่อสกัดเป็นสารสนเทศที่นำไปสนับสนุนการตัดสินใจได้จริง

1.2.5 ความท้าทายที่ผลักดันการพัฒนาเหมืองข้อมูล

Tan et al. (2006) ชี้ให้เห็นว่าข้อจำกัดของเครื่องมือวิเคราะห์ข้อมูลแบบดั้งเดิมต่อไปนี้เป็นแรงผลักดันสำคัญที่ทำให้ต้องพัฒนาเทคนิคการทำเหมืองข้อมูลขึ้นมา

ความสามารถรองรับข้อมูลขนาดใหญ่' (Scalability)
ชุดข้อมูลในยุคปัจจุบันมีขนาดตั้งแต่ระดับ Gigabytes ไปจนถึง Petabytes

อัลกอริทึมจึงต้องออกแบบให้รองรับข้อมูลขนาดใหญ่ได้ เช่น การใช้ Out-of-core Algorithms การสุ่มตัวอย่าง หรือการประมวลผลแบบขนาน

ข้อมูลหลายมิติ (High Dimensionality) ข้อมูลสมัยใหม่อาจมีคุณลักษณะ (Attribute) หลายร้อยหรือหลายพันตัว ซึ่งก่อให้เกิดปัญหาที่เรียกว่า Curse of Dimensionality ทำให้ต้องพัฒนาเทคนิคการคัดเลือกคุณลักษณะและการลดจำนวนมิติ

ข้อมูลที่หลากหลายและซับซ้อน (Heterogeneous and Complex Data) ข้อมูลในโลกจริงไม่ได้จำกัดอยู่เพียงตัวเลขหรือข้อความธรรมดา แต่ยังรวมถึงหน้าเว็บ ลำดับดีเอ็นเอ ข้อมูลภูมิสารสนเทศ และอนุกรมเวลา (Time Series) ซึ่งล้วนต้องการเทคนิคเฉพาะทาง

การวิเคราะห์ในรูปแบบใหม่ (Non-traditional Analysis) กระบวนทัศน์ดั้งเดิมที่อาศัยการตั้งสมมติฐานแล้วออกแบบการทดลองนั้นใช้ทรัพยากรมหาศาลและไม่ตอบโจทย์งานที่ต้องประเมินสมมติฐานจำนวนมากพร้อมกัน การทำเหมืองข้อมูลจึงมุ่งทำให้กระบวนการเหล่านี้เป็นไปโดยอัตโนมัติ

1.2.6 Generalization และ Overfitting: หัวใจสำคัญของการเรียนรู้

Bias-Variance Decomposition

$$E[(y - \hat{y})^2] = \text{Bias}^2(\hat{y}) + \text{Var}(\hat{y}) + \sigma^2$$

โดยที่:

$E[(y - \hat{y})^2]$ = ความคลาดเคลื่อนกำลังสองเฉลี่ย (Expected Mean Squared Error) ซึ่งเป็นตัวชี้วัดรวมของโมเดล

$$\text{Bias}^2(\hat{y}) =$$

อคติกำลังสองแสดงว่าโมเดลพยากรณ์ผิดพลาดอย่างเป็นระบบมากน้อยเพียงใด
โมเดลที่ซับซ้อนน้อยเกินไปมักมี Bias สูง

$$\text{Var}(\hat{y}) =$$

ความแปรปรวน แสดงว่าโมเดลไวต่อข้อมูลฝึกสอนมากน้อยเพียงใด

โมเดลที่ซับซ้อนเกินไปมักมี Variance สูง

σ^2 = ความคลาดเคลื่อนที่ลดไม่ได้ (Irreducible Error) เกิดจาก noise ในข้อมูล
ไม่สามารถลดลงได้ด้วยการปรับโมเดล

หลักการ: การสร้างโมเดลที่ดีต้องสมดุลระหว่าง Bias และ Variance เรียกว่า Bias-Variance Tradeoff

Bishop (2006)

เน้นย้ำว่าเป้าหมายที่แท้จริงของการเรียนรู้ของเครื่องไม่ใช่การจำข้อมูลที่มีอยู่ได้แม่นยำ แต่คือความสามารถในการ Generalize นั่นคือนำความรู้ที่ได้ไปทำนายข้อมูลใหม่ที่ไม่เคยพบมาก่อนได้อย่างถูกต้อง ความสามารถนี้คือเกณฑ์วัดคุณภาพของโมเดลที่แท้จริง

ปัญหา Overfitting เกิดขึ้นเมื่อโมเดลมีความซับซ้อนสูงเกินไป จนสามารถจำข้อมูลฝึก (Training Data) ได้อย่างสมบูรณ์ แต่กลับทำงานได้ไม่ดีกับข้อมูลใหม่ Bishop (2006) ใช้ตัวอย่างการประมาณเส้นโค้งด้วย Polynomial เพื่ออธิบายปรากฏการณ์นี้ กล่าวคือ Polynomial ดีกรีสูง เช่น $M = 9$ สามารถผ่านทุกจุดในข้อมูลฝึกได้พอดี แต่กลับสร้างเส้นโค้งที่แกว่งผิดปกติและไม่สามารถทำนายข้อมูลใหม่ได้อย่างสมเหตุสมผล

$$y(x, w) = \sum_{j=0}^M w_j x^j \text{ (Polynomial model)}$$

$$E(w) = \frac{1}{2} \sum_{n=1}^N (y(x_n, w) - t_n)^2 \text{ (Error function)}$$

แนวทางป้องกัน Overfitting ที่ใช้บ่อยในทางปฏิบัติ ได้แก่ การเลือกโมเดลที่มีความซับซ้อนเหมาะสม (Model Selection) การเพิ่มพจน์ลงโทษสำหรับโมเดลที่ซับซ้อนเกินไป (Regularization) การตรวจสอบไขว้ (Cross-Validation) และการเพิ่มขนาดของชุดข้อมูลฝึก ซึ่งจะกล่าวถึงในรายละเอียดในบทต่อไป

1.3 การเก็บรวบรวมข้อมูลจากไฟล์

ข้อมูลจำนวนมากในทางปฏิบัติถูกจัดเก็บในรูปแบบไฟล์

รูปแบบที่ได้รับความนิยมแต่ละประเภทมีลักษณะและการใช้งานที่แตกต่างกัน ดังนี้

CSV (Comma-Separated Values) จัดเก็บข้อมูลตารางในรูปแบบข้อความธรรมดา โดยใช้เครื่องหมายจุลภาคคั่นแต่ละค่า เป็นรูปแบบที่เรียบง่าย อ่านได้ด้วยโปรแกรมหลากหลาย และรองรับการใช้งานกว้างขวางที่สุด

TXT (Plain Text) ข้อความธรรมดาที่ไม่มีโครงสร้างตายตัว หากมีรูปแบบของตัวคั่นที่ชัดเจน ก็สามารถอ่านด้วย Pandas ได้เช่นเดียวกับไฟล์ CSV

XLSX (Microsoft Excel) รูปแบบไฟล์ของ Microsoft Excel ที่รองรับการมีหลายชีต สูตรคำนวณ และการจัดรูปแบบเซลล์ เหมาะกับข้อมูลที่ต้องการโครงสร้างซับซ้อน

JSON (JavaScript Object Notation) รูปแบบแลกเปลี่ยนข้อมูลที่มีน้ำหนักเบาและอ่านง่าย แสดงข้อมูลในลักษณะคู่คีย์-ค่า (Key-Value Pairs) นิยมใช้กับ Web API

XML (eXtensible Markup Language) จัดเก็บข้อมูลที่มีโครงสร้างลำดับชั้นโดยใช้แท็ก ออกแบบมาให้อธิบายตัวเองได้ เหมาะกับข้อมูลที่มีความสัมพันธ์แบบซ้อนกัน

HTML (Hypertext Markup Language) ภาษาสำหรับจัดโครงสร้างเนื้อหาบนเว็บ Pandas สามารถดึงตารางข้อมูลออกจากไฟล์ HTML ได้โดยตรง

1.3.1 แบบฝึกหัด — การนำเข้าข้อมูลจากไฟล์ด้วย Pandas

Pandas จัดเตรียมฟังก์ชันสำหรับนำเข้าข้อมูลจากไฟล์แต่ละรูปแบบไว้อย่างครบถ้วน โดยเริ่มต้นด้วยการ import ไลบรารี

```
import pandas as pd
```

1.3.1.1 CSV

ฟังก์ชัน `read_csv()` รองรับการจัดเก็บข้อมูลหลายรูปแบบ ในกรณีทั่วไปสามารถระบุเพียงเส้นทางไฟล์

```
df = pd.read_csv('/content/ds_salaries.csv')
```

หากต้องการปรับแต่งเพิ่มเติม เช่น ไม่ใช้แถวแรกเป็นหัวคอลัมน์ หรือข้ามบางแถว สามารถระบุอาร์กิวเมนต์ได้ดังนี้

```
df = pd.read_csv('/content/ds_salaries.csv', header=None, skiprows=1)
```

1.3.1.2 TXT

หากไฟล์ .txt มีโครงสร้างแบบ CSV สามารถอ่านด้วย read_csv() ได้โดยตรง

```
df = pd.read_csv('/content/ds_salaries.txt')
```

1.3.1.3 Excel

```
df = pd.read_excel('/content/ds_salaries.xlsx')
```

1.3.1.4 JSON

```
df = pd.read_json('/content/ds_salaries.json')
```

1.3.1.5 XML

```
df = pd.read_xml('/content/ds_salaries.xml')
```

1.3.1.6 HTML

```
df = pd.read_html('/content/ds_salaries.htm')[0]
```

1.4 การเก็บรวบรวมข้อมูลจากเว็บ

เว็บไซต์เป็นแหล่งข้อมูลขนาดใหญ่ที่อัปเดตอย่างต่อเนื่อง การดึงข้อมูลจากเว็บจึงมีคุณค่าในหลายมิติ ทั้งในแง่ของปริมาณข้อมูล ความทันสมัย และ ความหลากหลายของหัวข้อที่ครอบคลุม การเก็บรวบรวมข้อมูลจากเว็บมีประโยชน์ในด้านหลักดังนี้

การเข้าถึงข้อมูลจำนวนมาก เว็บคือคลังข้อมูลที่ใหญ่ที่สุดในโลก ครอบคลุมเกือบทุกหัวข้อ การนำข้อมูลเหล่านี้มาใช้ ช่วยสนับสนุนการตัดสินใจ การวิจัย และการวิเคราะห์ได้อย่างกว้างขวาง

ข้อมูลแบบเรียลไทม์ เว็บเป็นช่องทางหลักในการเผยแพร่ข้อมูลแบบทันที ไม่ว่าจะเป็นข่าวสาร ราคาตลาด หรือกิจกรรมบนโซเชียลมีเดีย

ข่าวกรองเชิงแข่งขัน ธุรกิจสามารถติดตามความเคลื่อนไหวของคู่แข่ง วิเคราะห์กลยุทธ์ และประเมินสภาพแวดล้อมของตลาดได้ผ่านการดึงข้อมูลจากเว็บ

การวิจัย และการวิเคราะห์
นักวิจัยและนักวิเคราะห์ข้อมูลใช้การดึงข้อมูลจากเว็บเพื่อสร้างชุดข้อมูลขนาดใหญ่ที่ไม่อาจรวบรวมด้วยวิธีอื่นได้

เว็บไซต์มีโครงสร้างที่หลากหลาย ซึ่งส่งผลต่อวิธีการดึงข้อมูลที่เหมาะสม แบ่งออกเป็น 3 ประเภทหลัก

เว็บไซต์ที่มีโครงสร้างชัดเจน (Structured Websites)
มีรูปแบบการจัดวางข้อมูลที่คงที่และเป็นระเบียบ เช่น ตารางข้อมูล สามารถดึงข้อมูลได้โดยตรงด้วย Pandas หรือไลบรารี BeautifulSoup และ lxml

เว็บไซต์กึ่งโครงสร้าง (Semi-Structured Websites)
มีทั้งส่วนที่มีโครงสร้างและส่วนที่ไม่มีโครงสร้างปะปนกัน ต้องใช้ Regular Expressions หรือการทำความสะอาดข้อมูลเพิ่มเติม

เว็บไซต์ที่ไม่มีโครงสร้าง (Unstructured Websites)
เนื้อหาอยู่ในรูปข้อความอิสระโดยไม่มีรูปแบบตายตัว การดึงข้อมูลต้องอาศัยเทคนิคขั้นสูง เช่น การประมวลผลภาษาธรรมชาติ (NLP) หรือการเรียนรู้ของเครื่อง

1.4.1 แบบฝึกหัด — การดึงข้อมูลจากเว็บ

1.4.1.1 Wikipedia

Wikipedia เป็นตัวอย่างเว็บไซต์ที่มีโครงสร้างชัดเจน สามารถดึงตารางข้อมูลออกมาได้โดยตรงด้วยฟังก์ชัน `pd.read_html()`

```
table = pd.read_html('https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)#Table')
df = table[2]
df.head()
```

1.4.1.2 Web Scraping

สำหรับเว็บไซต์ที่โครงสร้างที่ไม่สามารถดึงข้อมูลด้วย `read_html()` ได้โดยตรง ต้องใช้วิธี Web Scraping โดยอ่านและแยกวิเคราะห์โค้ด HTML ของหน้าเว็บ ทั้งนี้ผู้ใช้ควรศึกษาพื้นฐาน HTML และ XML ก่อน และตรวจสอบว่าเว็บไซต์นั้นอนุญาตให้ทำ Scraping หรือไม่

ขั้นแรก ดาวน์โหลดหน้าเว็บด้วยไลบรารี `requests`

```
import requests

page = requests.get('https://dataquestio.github.io/web-scraping-pages/simple.html')
```

`page.status_code` # ค่า 200 หมายถึงสำเร็จ

จากนั้นแยกวิเคราะห์โครงสร้าง HTML ด้วย `BeautifulSoup`

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(page.content, 'html.parser')

print(soup.prettify())
```

ค้นหาแท็กที่ต้องการด้วย `find_all()` สำหรับทุกอินสแตนซ์ หรือ `find()` สำหรับอินสแตนซ์แรก นอกจากนี้ยังกรองตาม `class` หรือ `id` ได้

```
soup.find_all('title')
```

```
soup.find_all('p', class_='outer-text')
```

```
soup.find_all(id='first')
```

1.5 การเก็บรวบรวมข้อมูลจากฐานข้อมูล SQL

ฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ที่ใช้ภาษา SQL เป็นตัวจัดการนั้น เป็นระบบจัดเก็บข้อมูลหลักขององค์กรมาอย่างยาวนาน เพราะมีคุณลักษณะที่เหมาะสมกับการจัดการข้อมูลปริมาณมากหลายประการ

โครงสร้างที่ชัดเจน ข้อมูลถูกจัดเก็บในรูปตาราง (Table) ที่ประกอบด้วยแถว (Row) และคอลัมน์ (Column) ซึ่งทำให้ค้นหาและดึงข้อมูลได้อย่างมีประสิทธิภาพ

ความถูกต้องและสอดคล้องของข้อมูล ฐานข้อมูล SQL บังคับใช้กฎความสมบูรณ์ผ่านข้อจำกัดต่างๆ เช่น Primary Key, Unique Key และ Referential Integrity เพื่อป้องกันข้อมูลที่ผิดพลาดหรือซ้ำซ้อน

ภาษา SQL ที่ทรงพลัง SQL ช่วยให้ได้ถึงกรองรวม และวิเคราะห์ข้อมูลที่ซับซ้อนได้ด้วยคำสั่งเพียงไม่กี่บรรทัด

คุณลักษณะ ACID ฐานข้อมูล SQL รับประกันว่าทุกการดำเนินการเป็นไปอย่างถูกต้อง (Atomicity) สอดคล้องกัน (Consistency) แยกกัน (Isolation) และคงทน (Durability)

Python มีไลบรารีเชื่อมต่อฐานข้อมูลหลายตัวให้เลือกใช้ตามความเหมาะสม เช่น sqlite3 สำหรับ SQLite, psycopg2 สำหรับ PostgreSQL, pymysql สำหรับ MySQL และ pyodbc สำหรับฐานข้อมูลผ่าน ODBC

1.5.1 แบบฝึกหัด — การดึงข้อมูลจาก SQLite

SQLite เป็นฐานข้อมูลขนาดเล็กที่จัดเก็บข้อมูลทั้งหมดในไฟล์เดียว เหมาะอย่างยิ่งสำหรับการศึกษาและพัฒนาโปรแกรมขนาดเล็ก ขั้นตอนการทำงานมีดังนี้

1. สร้าง connection object เพื่อเชื่อมต่อกับไฟล์ฐานข้อมูล
2. สร้าง cursor object สำหรับส่งคำสั่ง SQL
3. เขียนและดำเนินการ Query
4. ดึงผลลัพธ์มาประมวลผล
5. ปิดการเชื่อมต่อเมื่อเสร็จสิ้น

```
import sqlite3
```

```
connection = sqlite3.connect('/content/ds_salaries.sqlite')
```

```
cursor = connection.cursor()
```

```
query = 'SELECT name FROM sqlite_master WHERE type="table";'
```

```
cursor.execute(query)
```

```
results = cursor.fetchall()
```

หลังจากดึงข้อมูลได้แล้ว สามารถแปลงเป็น Pandas DataFrame และบันทึกเป็นไฟล์ CSV ได้ดังนี้

```
df = pd.DataFrame(results, columns=cols)
```

```
df.to_csv('/content/output.csv')
```

```
cursor.close()
```

```
connection.close()
```

1.5.2 กรณีศึกษา — การดึงข้อมูลการช้อปปิ้งจาก SQLite

กรณีศึกษา นี้ สาธิตการทำงานกับฐานข้อมูล shopping.sqlite ซึ่งบรรจุข้อมูลการช้อปปิ้งจำลองทั้งหมด 16,029 รายการ โดยแต่ละรายการมีคอลัมน์ดังนี้

invoice_no, customer_id, gender, age, category, quantity, price,
payment_method, invoice_date และ shopping_mall

connection = sqlite3.connect('/content/shopping.sqlite')

query = 'SELECT * FROM customer_shopping_data'

cursor.execute(query)

results = cursor.fetchall()

df = pd.DataFrame(results, columns=cols)

df.to_csv('/content/shopping.csv')

1.6 การเก็บรวบรวมข้อมูลผ่าน API

API (Application Programming Interface)

คือช่องทางมาตรฐานที่ผู้ให้บริการข้อมูลเปิดให้ผู้พัฒนาเข้าถึงข้อมูลหรือบริการของตนได้อย่างเป็นระบบ การเก็บรวบรวมข้อมูลผ่าน API มีข้อได้เปรียบสำคัญหลายประการ

ข้อมูลที่มีโครงสร้างมาตรฐาน API ส่งคืนข้อมูลในรูปแบบที่กำหนดไว้ชัดเจน เช่น JSON หรือ XML ทำให้นำไปประมวลผลต่อได้ทันทีโดยไม่ต้องผ่านการทำความสะอาดมาก

ข้อมูลทันสมัยและเชื่อถือได้ API มักส่งข้อมูลแบบเรียลไทม์หรือใกล้เคียงเรียลไทม์จากแหล่งที่เชื่อถือได้โดยตรง

การควบคุมการเข้าถึง ผู้ให้บริการสามารถกำหนดสิทธิ์ ขีดจำกัดการใช้งาน และกลไกการพิสูจน์ตัวตน เพื่อป้องกันการใช้งานในทางที่ผิด

ความสะดวกในการทำงานอัตโนมัติ การเรียก API สามารถตั้งเวลาหรือฝังอยู่ในกระบวนการทำงานอัตโนมัติได้ ช่วยให้แอปพลิเคชันได้โดยไม่ต้องดำเนินการด้วยมือ

ตัวอย่าง API ที่ใช้อยู่ในงานวิเคราะห์ข้อมูล

Yahoo Finance API สำหรับข้อมูลตลาดการเงิน ราคาหุ้น และข้อมูลบริษัทจดทะเบียน

OpenWeatherMap API สำหรับข้อมูลสภาพอากาศปัจจุบัน พยากรณ์อากาศ และข้อมูลอุตุนิยมวิทยาย้อนหลัง

Twitter/X API สำหรับดึงโพสต์และข้อมูลผู้ใช้ เพื่อนำไปวิเคราะห์ความรู้สึก (Sentiment Analysis) หรือติดตามเทรนด์

Google Maps API สำหรับบริการเชิงตำแหน่ง เช่น Geocoding การคำนวณเส้นทาง และการแสดงผลแผนที่

1.6.1 แบบฝึกหัด — การดึงข้อมูลจาก Yahoo Finance

แบบฝึกหัดนี้สาธิตการดึงข้อมูลทางการเงินจาก Yahoo Finance โดยใช้ Ticker Symbol ของบริษัท เช่น AMZN สำหรับ Amazon และ GOOGL สำหรับ Google
ขั้นแรก ติดตั้งไลบรารีที่จำเป็น

```
!pip install yfinance
```

```
!pip install yahoofinancials
```

นำเข้าไลบรารีและดึงข้อมูลของบริษัท

```
import yfinance as yahooFinance
```

```
GoogleInfo = yahooFinance.Ticker('GOOGL')
```

```
print(GoogleInfo.info)
```

เลือกดึงเฉพาะข้อมูลที่ต้องการ เช่น ภาคธุรกิจ อัตราส่วน P/E และค่า Beta

```
print('Company Sector:', GoogleInfo.info['sector'])
```

```
print('Price Earnings Ratio:', GoogleInfo.info['trailingPE'])
```

```
print('Company Beta:', GoogleInfo.info['beta'])
```


ดึงราคาหุ้นย้อนหลัง หรือดึงหลายหุ้นพร้อมกันในช่วงเวลาที่กำหนด

```
print(GoogleInfo.history(period='max'))
```

```
df = yahooFinance.download('AMZN GOOGL', start='2019-01-01', end='2020-01-01', group_by='ticker')
```

บันทึกข้อมูลเป็นไฟล์ CSV

```
df.to_csv('data/FinanceData.csv')
```

เอกสารอ้างอิง

Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction (2nd ed.). Springer.

Tan, P. N., Steinbach, M., & Kumar, V. (2006). Introduction to Data Mining. Pearson Addison Wesley.

Wu, D. (2024). Data Mining with Python: Theory, Application, and Case Studies. CRC Press, Taylor & Francis Group.